



AULA 3

Recursos avançados aplicados ao aprendizado supervisionado



Objetivos de aprendizagem

Ao final desta aula, você irá:

- Entender o funcionamento dos principais algoritmos de *ensembles* utilizados em problemas de classificação e regressão.
- Compreender o conceito e os principais métodos de *feature selection* e transformação de dados em *machine learning*.
- Compreender o funcionamento das bibliotecas de *pipelines* e otimização de hiperparâmetros.
- Construir na prática um modelo de classificação utilizando conceitos avançados.

Que história é essa?

Nas aulas anteriores desta disciplina, estudamos o aprendizado supervisionado em *machine learning* e exploramos os problemas de classificação e regressão.

Nesta aula, falaremos sobre alguns conceitos e recursos avançados. Veremos o que são *ensembles* e estudaremos o funcionamento dos principais algoritmos de *ensembles* utilizados em *machine learning*. Também iremos compreender o conceito e explorar os principais métodos de *feature selection* (seleção de características) e transformação de dados em *machine learning*, bem como o

funcionamento das bibliotecas de *pipelines* e otimização de hiperparâmetros para algoritmos de *machine learning*.

Finalmente, iremos construir na prática um modelo de classificação utilizando os conceitos avançados aprendidos nesta aula.

Ensembles

Os métodos *ensemble* (ou comitês) combinam vários modelos de *machine learning*, o que pode fazer com que os resultados sejam melhores do que quando usamos apenas um modelo. Muitas vezes, esses modelos têm melhor desempenho preditivo em comparação com um único modelo, sendo os primeiros colocados em muitas competições de aprendizado de máquina de prestígio, como as do Kaggle.

Conceitualmente, *ensembles* são meta-algoritmos que combinam várias técnicas de aprendizado de máquina em um modelo preditivo, com o objetivo de diminuir a variância, o viés ou melhorar as predições, podendo ser de dois grupos. Clique nas abas para conhecê-los:

MÉTODOS SEQUENCIAIS



Os modelos base são gerados sequencialmente e o *ensemble* explora a dependência entre os modelos de base. O desempenho geral pode ser aprimorado ponderando os exemplos previamente rotulados incorretamente com maior peso.

MÉTODOS PARALELOS



Os modelos base são gerados em paralelo e o *ensemble* explora a independência entre os modelos base. A predição final é uma consolidação da predição dos modelos base.

A maioria dos métodos *ensemble* usa um único algoritmo básico, produzindo *ensembles* **homogêneos**, mas também há métodos que usam algoritmos diferentes, os *ensembles* **heterogêneos**. Para que o *ensemble* seja melhor do que qualquer um de seus modelos individuais, os modelos base precisam ter boa precisão e ser diversificados, idealmente, com alta variância e baixo viés (*bias*). Conheça os métodos *ensemble* mais populares.

Interativo

Descrição do interativo

Agora, conheça mais detalhes de cada um desses métodos.

Voting

Os *ensembles* do tipo *voting* utilizam uma das técnicas mais simples de combinar previsões de vários modelos. Os *ensembles* dessa categoria criam dois ou mais modelos independentes a partir de todo o conjunto de dados de treinamento. A previsão final geralmente é feita por votação majoritária (para problemas de classificação) ou média (para problemas de regressão).

No *ensemble* do tipo *voting*, as previsões dos submodelos podem ser ponderadas, mas é difícil especificar os pesos dos classificadores manualmente ou mesmo heurísticamente.

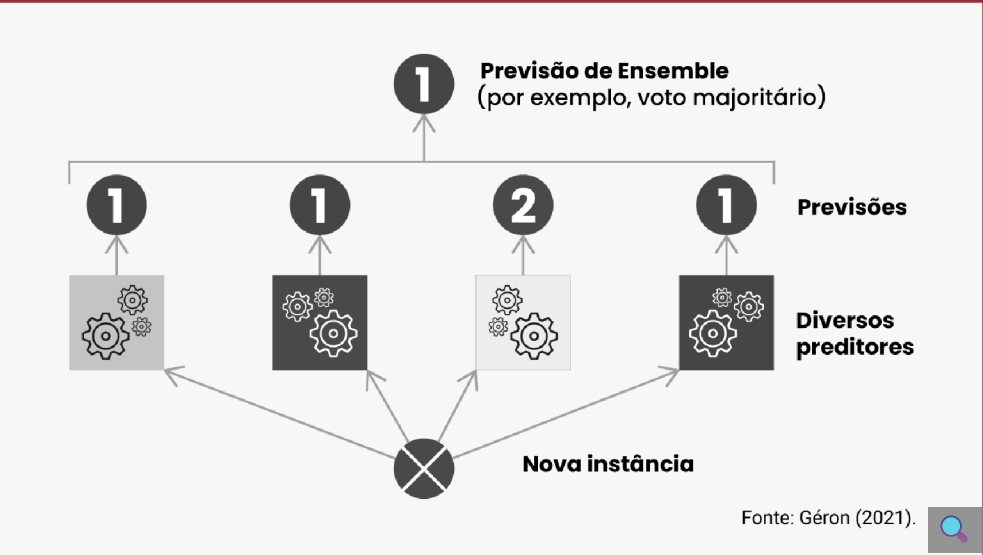


Aprenda mais

Existem métodos mais avançados que podem aprender a ponderar as previsões dos submodelos construindo um novo modelo. Essa técnica é conhecida como **stacking (stacked aggregation)** e está fora do escopo desta disciplina.

Em problemas de classificação, a classe predita pelo *ensemble* é a mais votada entre os classificadores. O *ensemble* geralmente tem melhor acurácia que o melhor classificador do *ensemble*, e mesmo que todos os classificadores sejam “fracos” (um pouco melhores que escolha aleatória), o *ensemble* ainda pode ser um classificador “forte”, desde que os classificadores sejam numerosos e diversos o suficiente.

A figura a seguir ilustra a decisão de um *ensemble* do tipo *voting* para problemas de classificação. Repare que três dos quatro modelos base tiveram como saída a classe 1, e o quarto modelo teve como saída a classe 2. Como a classe 1 teve três dos quatro votos, ela é a resposta do *ensemble* para o problema.



Recomenda-se usar *ensembles* do tipo *voting* quando todos os modelos do *ensemble* tiverem, em geral, boa performance e quando a maioria dos modelos concordar entre si. Se qualquer modelo base do *ensemble* tiver um desempenho melhor que o *ensemble* em si, esse modelo provavelmente deverá ser usado em vez do *ensemble*.



Fique ligado!

Também podemos usar *voting ensembles* para combinar diferentes versões de um mesmo algoritmo de *machine learning*, variando os seus hiperparâmetros.

Bagging

Em *ensembles* do tipo *bagging*, são construídos vários modelos (geralmente do mesmo tipo de algoritmo) a partir de diferentes subamostras do conjunto de dados de treinamento.

Para tal, é utilizada a técnica de **Bootstrap**, um método estatístico poderoso para estimar uma quantidade (como média ou desvio padrão) a partir de uma amostra pequena de dados.

Vejamos um exemplo:

Imagine que temos uma amostra de 100 valores (x_1 a x_{100}) e gostaríamos de obter uma estimativa da média dessa amostra. Sabemos que é possível calcular a média diretamente da amostra como:

$$média(x) = \frac{1}{100} \times \sum_{i=1}^{100} x_i$$

Entretanto, como sabemos que a amostra é pequena, essa média provavelmente tem um erro. Podemos melhorar a estimativa de nossa média usando o Bootstrap. Para tal:

1. Crie muitas subamostras aleatórias (por exemplo, 1.000) do conjunto de dados com substituição.



2. Calcule a média de cada subamostra.

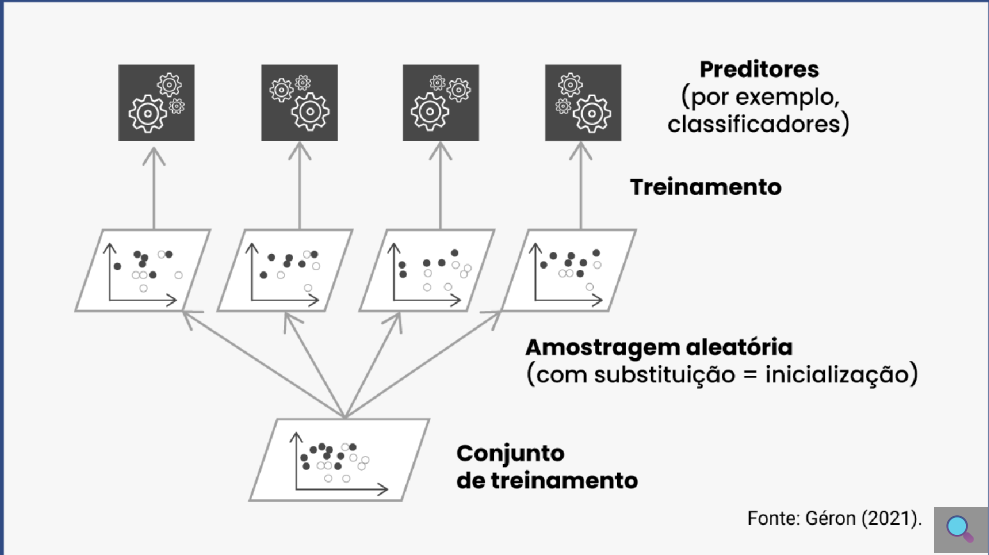


3. Calcule a média das médias coletadas e use-a como a média estimada para os dados.

Usando esse racional, os *ensembles* do tipo *bagging* utilizam o mesmo algoritmo para cada modelo base, treinando cada um deles em um subconjunto aleatório do conjunto de treino.

Em geral, esse tipo de *ensemble* trabalha com árvores de decisão como algoritmo dos seus modelos base. Após o treinamento dos modelos base, a predição do *ensemble* para uma nova instância é a agregação das predições individuais, geralmente pela classe mais votada (para problemas de classificação) ou média das saídas (para problemas de regressão).

A figura a seguir ilustra o treinamento de um *ensemble* do tipo *bagging* para problemas de classificação. Repare que, a partir do conjunto de treino, foram construídas quatro subamostras (usando Bootstrap, ou seja, com reposição) e cada uma delas foi utilizada para treinar um modelo base distinto.



Uma forma de reduzir a variância de uma predição é calcular a média (ou moda) de várias predições. Os *ensembles* do tipo *bagging* fazem isso, pois consolidam as predições de várias subamostras do conjunto de treinamento. Assim, esses *ensembles* podem ser usados para reduzir a variância dos modelos base com alta variância (por exemplo, árvores de decisão, que são muito sensíveis aos dados específicos que são treinados). Como utiliza apenas um subamostra, cada modelo base tem maior *bias* do que se fosse treinado com todo o conjunto de treinamento, mas a agregação reduz tanto o *bias* quanto a variância do *ensemble*.

O funcionamento dos *ensembles* do tipo *bagging* pode ser resumizado por:

1. Crie várias subamostras (com aproximadamente 60% do *dataset* de treinamento original) aleatórias do conjunto de dados de treino, com reposição.
2. Escolha um algoritmo (por exemplo, árvore de decisão) e treine um modelo de *machine learning* em cada subamostra.
3. Dada uma nova instância de dados, calcule a predição do *ensemble* (para problemas de classificação, a classe mais frequente; para problemas de regressão, a média das saídas de cada modelo individual).

O número de subamostras a serem geradas e, conseqüentemente, o número de modelos base a serem criados é um parâmetro do *ensemble*, que pode ser escolhido experimentalmente.



Aprenda mais

Um exemplo muito conhecido dos *ensembles* do tipo *bagging* é o algoritmo **Random Forest** (ou Floresta Aleatória), que representa uma melhoria em relação ao *bagging* tradicional com árvores de decisão. Esse algoritmo também utiliza diferentes subamostras aleatórias do conjunto de treinamento para a criação de modelos base. Além disso, durante a etapa de treinamento de cada um desses modelos, é utilizado um subconjunto aleatório dos atributos para decidir como fazer a divisão da árvore em cada nó, procurando o melhor atributo dentre a subamostra de atributos. Para essa amostra de atributos, o algoritmo busca o ponto de corte ótimo para cada atributo. Isso resulta em maior diversidade de árvores, geralmente produzindo um *ensemble* melhor.

Outro conhecido exemplo de *ensemble* do tipo *bagging* é o algoritmo **Extra Trees**, também conhecido como Extremely Randomized Trees (árvores extremamente aleatórias), que é uma extensão do Random

Forest. Diferentemente do *bagging* e do Random Forest (que constroem cada árvore de decisão a partir de uma amostra Bootstrap do conjunto de dados de treinamento), o Extra Trees ajusta cada árvore de decisão utilizando todo o conjunto de dados de treinamento.



Para o treinamento de cada árvore de decisão base, o Extra Trees utiliza um subconjunto aleatório de atributos para avaliar as divisões. Essas divisões são realizadas aleatoriamente, ou seja, a escolha da divisão a realizar é a melhor divisão usando pontos de corte aleatórios em um subconjunto de atributos selecionado aleatoriamente para o nó da árvore corrente.

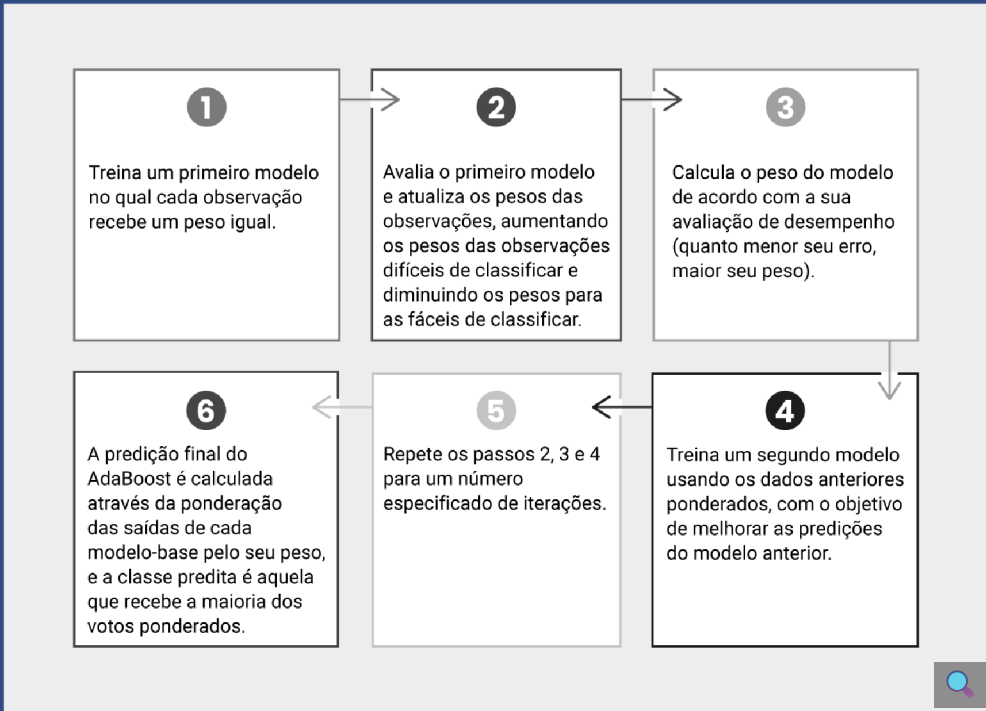
Devido a essa grande aleatoriedade, é um modelo mais rápido computacionalmente do que o Random Forest, pois não é necessário avaliar onde realizar cada divisão.

Boosting

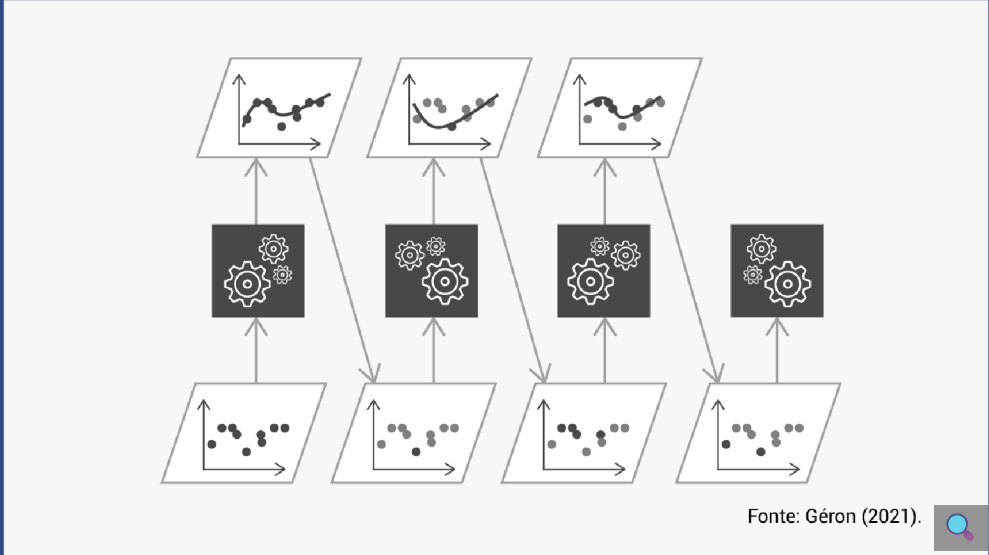
Os *ensembles* do tipo *boosting* são ***ensembles sequenciais***. Eles criam uma sequência de modelos na qual um modelo tenta corrigir os erros do modelo anterior. Esse tipo de *ensemble* busca converter modelos fracos em modelos fortes e é do tipo homogêneo, sendo geralmente utilizado com árvores de decisão.

O **AdaBoost** foi o primeiro algoritmo de *boosting* bem-sucedido e é o mais popular. Ele pondera as instâncias no conjunto de dados de acordo com o quão fácil ou difícil elas são classificadas no modelo corrente, permitindo que o algoritmo preste mais atenção nos exemplos “mais difíceis” na construção de modelos subsequentes. Para tal, maior peso é dado aos exemplos que foram classificados incorretamente no modelo anterior.

O funcionamento do AdaBoost para problemas de classificação pode ser resumizado por:



A figura a seguir ilustra o treinamento de um *ensemble* do tipo *boosting* que utiliza o algoritmo AdaBoost para problemas de regressão.



Repare que inicialmente um modelo é treinado e utilizado para fazer previsões no conjunto de treinamento. Em seguida, o peso relativo das instâncias de treinamento que tiveram a previsão incorreta é aumentado. Um segundo modelo é treinado usando os pesos atualizados, e as previsões são novamente realizadas no conjunto de treinamento. Os pesos das instâncias são atualizados, e assim sucessivamente.



Aprenda mais

Outro algoritmo de *boosting* muito popular é o **Gradient Boosting**, que serve de base para a implementação de sua versão otimizada, conhecida como **XGBoost** (Extreme Gradient Boosting). O Gradient Boosting é uma variação do AdaBoost que tenta ajustar o modelo-base novo aos erros residuais cometidos pelo modelo-base anterior, em vez de ajustar os pesos das observações a cada iteração. Para tal, o Gradient Boosting usa o procedimento de descida do gradiente (baseado em erros residuais, método cujo detalhamento está fora do escopo desta disciplina) para adicionar novos modelos-base, e o novo modelo reduz a perda, indo na direção do gradiente. É uma das técnicas de *ensemble* mais sofisticadas e uma das melhores técnicas disponíveis para melhorar o desempenho via *ensembles*.

Feature selection

Até agora, nesta disciplina, discutimos diversos algoritmos de *machine learning* aplicados a problemas de classificação e regressão e vimos que os *ensembles* são algoritmos que combinam diversos modelos com o objetivo de termos resultados melhores. Entretanto, os atributos (X) dos dados usados para treinamento dos modelos de *machine learning* têm enorme influência no seu desempenho. Atributos irrelevantes ou parcialmente relevantes podem afetar negativamente o resultado do modelo, especialmente em algoritmos lineares como regressão linear e regressão logística.

Assim, quando trabalhamos em problemas de *machine learning*, é importante prestarmos atenção no processo de seleção de atributos (características), conhecido como *feature selection*.



Mantendo o foco

Observe que esse processo antecede a etapa de modelagem e inferência do esquema básico de um projeto de ciência de dados.

Conforme visto na primeira aula desta disciplina, o processo consiste em selecionar os atributos (X) que mais contribuem para a saída (y) e utilizá-los para treinamento do modelo, ou seja, reduzir o número de variáveis de entrada do modelo.

Entre as possíveis vantagens de realizar a etapa de *feature selection*, podemos citar:

- (I) A redução de *overfitting*, pois, com dados menos redundantes, temos menor risco de tomar decisões com base em ruído;
- (II) A melhoria do resultado do modelo, pois dados menos “enganosos” e mais relevantes para o problema trazem melhor precisão do modelo;
- (III) Redução do tempo de treinamento, pois, quando trabalhamos com menos dados, temos um treinamento mais rápido;
- (IV) melhoria da interpretabilidade do modelo, pois, com menos atributos, há mais chances de melhor entendimento do processo que gerou os dados e do resultado gerado pelo modelo.

Neste ponto, é importante destacarmos a distinção entre *feature selection* e redução de dimensionalidade. Ambos os métodos tendem a reduzir o número de atributos (colunas) no conjunto de dados. Porém, os métodos de redução de dimensionalidade criam combinações dos atributos (também conhecido como *feature transformation*) usando operações de álgebra linear, enquanto os métodos de seleção de atributos excluem atributos do dataset (*feature selection*). São exemplos de métodos de redução de dimensionalidade o *Principal Component Analysis* (PCA) e *Singular Value Decomposition* (SVD).

- Os métodos de *feature selection* podem ser:
- **Não supervisionados**, quando não utilizam a variável de saída (y), por exemplo, removendo atributos duplicados ou redundantes na etapa de pré-processamento de dados.
 - **Supervisionados**, quando utilizam a variável de saída (y), por exemplo, removendo atributos irrelevantes para o problema, o que é possível fazer através de técnicas distintas.
- Conheça como funcionam essas técnicas, clicando sobre o título.

Seleção univariada	↓
Na seleção univariada, testes estatísticos são usados para selecionar os atributos que tenham relação mais forte com a variável de saída.	
Eliminação recursiva de atributos	↓
Na eliminação recursiva de atributos, os atributos são removidos recursivamente e vai sendo construído um modelo	

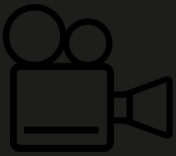
com os que permanecem, usando o resultado do modelo (por exemplo, a acurácia) para identificar quais atributos mais contribuem para a predição da variável de saída.

Importância de atributos



Já na importância de atributos, são utilizados algoritmos de *machine learning* (como Random Forest ou Extra Trees) para estimar a importância dos atributos, fazendo uma pré-seleção. É possível também realizar a operação de *feature selection* de forma intrínseca, através de algoritmos que fazem a seleção de atributos automaticamente como parte do treinamento do modelo (por exemplo, regressão linear com regularização Lasso e árvores de decisão).

É importante ressaltar que, assim como não há um “melhor” algoritmo de *machine learning*, não existe uma “melhor” técnica de *feature selection*. É preciso avaliar experimentalmente a técnica que produz os melhores resultados para cada problema específico.



Exemplo de *feature selection* em Python

Assista ao vídeo a seguir para ver alguns exemplos de código de métodos de *feature selection* e entender como eles funcionam na prática.



Transformação de dados: normalização e padronização

Muitos algoritmos de *machine learning* apresentam melhor resultado quando os atributos de entrada numéricos são redimensionados, por exemplo, em algoritmos que trabalham com a soma ponderada de entradas, como a regressão linear e a regressão logística, e em algoritmos baseados em distâncias, como o KNN, ou produto interno dos atributos de entrada, como o SVM.

As duas técnicas mais populares para redimensionar dados numéricos antes da modelagem são a **normalização**, que dimensiona cada atributo separadamente entre 0 e 1, e a **padronização**, que dimensiona cada atributo separadamente subtraindo a média e dividindo pelo desvio padrão, transformando sua distribuição em uma distribuição normal padrão, com média 0 e desvio padrão 1.

Já sabemos que os algoritmos de *machine learning* de aprendizado supervisionado mapeiam os atributos de entrada em uma variável de saída. Entretanto, a escala e a distribuição dos dados podem ser

diferentes para cada atributo do *dataset*, bem como estar em diferentes unidades. Essas diferenças podem aumentar a dificuldade do problema a ser modelado. Além disso, atributos de entrada com valores muito grandes podem resultar em um modelo que aprende pesos com valores altos, o que pode tornar o modelo instável e com *performance* de aprendizado ruim, pois pode ser muito sensível às diferenças dos valores de entrada, podendo resultar em alto erro de generalização.

É importante observar que a diferença na escala dos atributos não afeta todos os algoritmos de *machine learning*. Geralmente, os algoritmos baseados em árvores (como árvores de decisão ou Random Forest) não são afetados.

A seguir, veremos mais detalhes sobre a **normalização** e a **padronização**.

Interativo

Descrição do interativo

Pipelines

As operações de *feature selection* e transformação de dados fazem parte da etapa de **pré-processamento de dados** do esquema básico de um projeto de ciência de dados (visto na primeira aula desta disciplina). Esta costuma ser a etapa mais demorada e trabalhosa do projeto e consiste na preparação dos dados brutos em um formato apropriado para a modelagem. Na etapa de pré-processamento de dados, pode ser necessário, por exemplo, tratar dados faltantes e dados discrepantes (*outliers*), transformar dados e selecionar as características mais adequadas para construir os modelos na etapa seguinte, de modelagem e inferência.

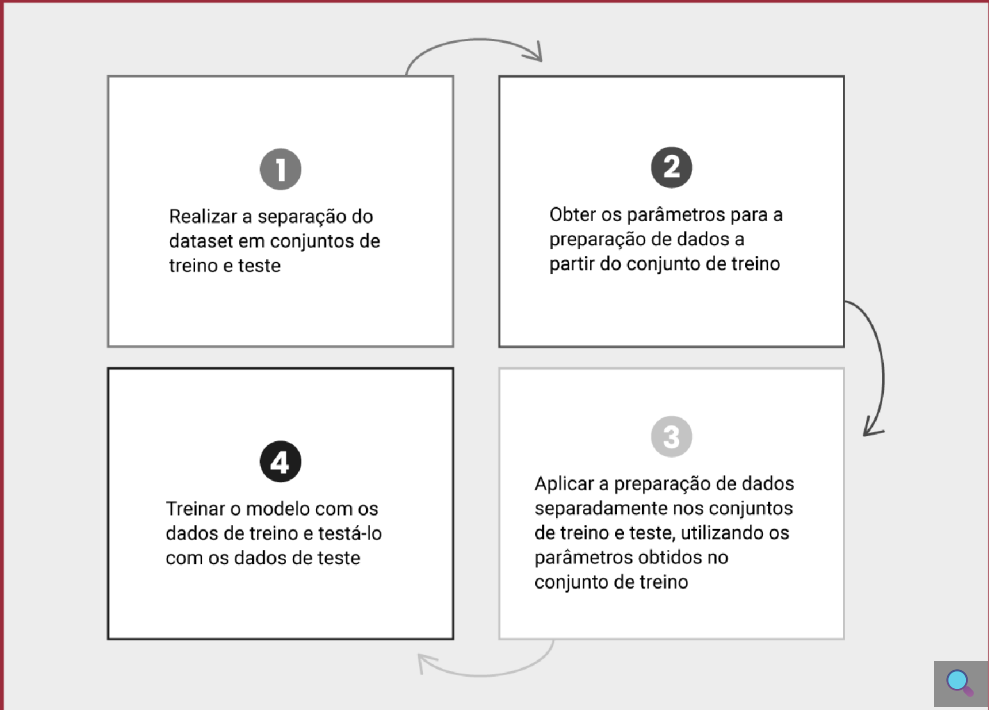
Realizar a preparação de dados é uma tarefa que deve ser realizada com atenção e cuidado, de forma a evitar um problema conhecido como ***data leakage*** (vazamento de dados). Esse problema ocorre quando há “vazamento” dos dados da base de teste para a base de treino, o que pode resultar em uma estimativa incorreta do desempenho do modelo ao realizar predições nos novos dados.

Ou seja: é como se na fase de treinamento do modelo fossem reveladas a ele, de forma antecipada e indevida, as informações “futuras” da base de teste. Por exemplo, preparar os dados usando normalização em todo o *dataset* antes do treinamento do modelo não seria recomendado, porque, para realizar essa transformação, calcularíamos os valores mínimo e máximo de todo o *dataset*, ou seja, o conjunto de treino normalizado seria influenciado pelo intervalo dos dados do conjunto de teste, uma vez que o conjunto de treino teria sido ajustado usando os valores mínimo e máximo globais. Apesar de esse vazamento ser geralmente sutil, ele pode interferir nos resultados do modelo



Este problema pode acontecer tanto quando validamos nossos modelos com a técnica de *holdout* (dividindo o *dataset* em conjuntos de treino e teste) como quando utilizamos a validação cruzada, pois ambas as técnicas consistem em treinar o modelo com uma parte dos dados e testá-lo com uma parte diferente dos dados, que não deve ser utilizada para o treinamento do modelo.

Para evitar a ocorrência de *data leakage*, precisamos garantir uma forte separação dos conjuntos de treino e teste, incluindo na etapa de preparação de dados. Assim, os parâmetros utilizados para a preparação de dados devem ser extraídos apenas do conjunto de treino e aplicados no conjunto de treino, e, uma vez ocorrido o treinamento do modelo, a preparação de dados do conjunto de teste deve utilizar os mesmos parâmetros utilizados no conjunto de treino. Para tal, deve-se:



Para nos ajudar nessa tarefa e evitar a ocorrência de *data leakage*, a biblioteca scikit-learn fornece um recurso conhecido como ***pipeline*** para ajudar a automatizar os fluxos de trabalho de *machine learning*. De forma resumida, os *pipelines* permitem que uma sequência linear de operações de preparação de dados seja encadeada com o treinamento do modelo, permitindo que este seja avaliado em seguida. O objetivo é garantir que todas as etapas do *pipeline* sejam restritas ao conjunto de dados apropriado (como o conjunto de treino do *holdout* ou cada *fold* do procedimento de validação cruzada), permitindo obter uma estimativa justa do desempenho do modelo com dados não vistos.



Mantendo o foco

De forma mais específica, todo o *pipeline* de modelagem (que inclui a preparação de dados e o treinamento do modelo) deve ser realizado apenas no conjunto de treinamento para evitar o *data leakage*. Para avaliar o desempenho desse modelo, as etapas utilizadas no *pipeline* de treino devem ser replicadas para o conjunto de teste, mas com os parâmetros obtidos com o conjunto de treino.

Otimização de hiperparâmetros

Após uma primeira avaliação, treinando modelos com os algoritmos de *machine learning* possíveis para tratar um problema e analisando os resultados obtidos, é interessante realizarmos o *tuning* (ajuste) dos melhores algoritmos encontrados até então, a fim de buscarmos construir o melhor modelo para o problema.

Para tal, os algoritmos de Machine Learning podem ser parametrizados com os chamados **hiperparâmetros**, a fim de que seu comportamento possa ser ajustado para um determinado problema. Os algoritmos podem ter diversos hiperparâmetros, que podem receber diversos valores para construir o modelo, e muitas vezes encontrar a melhor combinação de hiperparâmetros manualmente pode ser uma tarefa difícil.

Clique nas abas para ver alguns exemplos de hiperparâmetros para os algoritmos que estudamos nesta disciplina:

KNN	↓
Número de vizinhos a considerar (<i>k</i>) e métrica de distância utilizada;	
Árvore de decisão	↓
Profundidade máxima da árvore e critério de avaliação da qualidade dos particionamentos;	
SVM	↓
Tipo de <i>kernel</i> utilizado e rigidez da margem;	
Regressão Linear Ridge e Lasso	↓
Impacto da penalidade da regularização;	
Bagging	↓
Algoritmo dos modelos base e número de membros do <i>ensemble</i> .	

Para nos ajudar na tarefa da otimização de hiperparâmetros, a biblioteca scikit-learn fornece uma funcionalidade conhecida como ***grid search***, que nos permite informar em um *grid* quais

hiperparâmetros queremos variar, bem como o conjunto de valores que queremos avaliar para cada um deles. Para cada combinação de hiperparêmetros especificada no *grid*, a biblioteca irá sistematicamente construir e avaliar um modelo utilizando validação cruzada. Isso nos permitirá escolher, ao final desse processo, a melhor combinação de hiperparâmetros para o algoritmo em questão.

Finalmente, poderemos preparar o modelo escolhido para ser implantado, treinando-o com todo o *dataset* disponível.



Reflita!

Quais os principais desafios relacionados a projetos de *machine learning* que você encontraria na sua empresa?

Prática avançada de *machine learning* em Python (classificação)

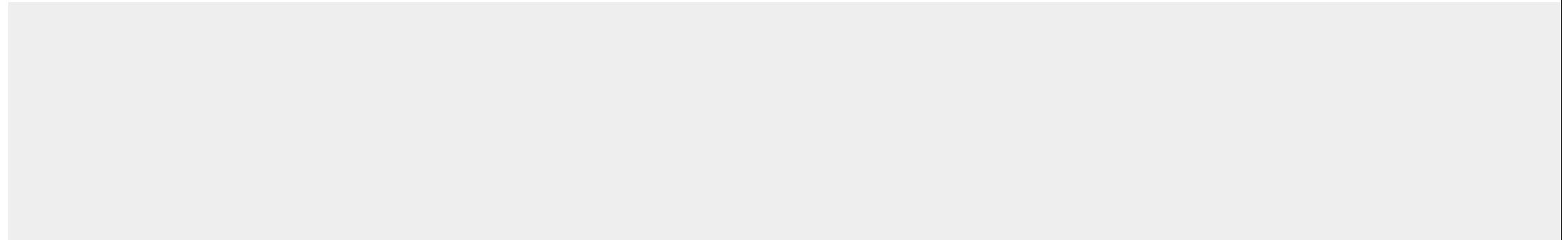
Para ilustrar alguns dos conceitos que estudamos nesta aula na prática, vamos ver um exemplo de código. Para esta prática, vamos utilizar o *dataset* **Pima Indians Diabetes**, que é originalmente do Instituto Nacional de Diabetes e Doenças Digestivas e Renais.

Seu objetivo é prever se um paciente terá ou não diabetes com base em certas medidas de diagnóstico médico. Este é um subconjunto de um *dataset* maior, e aqui todos os pacientes são mulheres com pelo menos 21 anos de idade e de herança indígena pima.

Esse *dataset* consiste em vários atributos de exames médicos e uma variável de classe. Vejamos no que consistem os atributos desse *dataset*:

- **Pregnancies**: números de vezes que a mulher já engravidou
- **Glucose**: concentração de glicose no plasma
- **BloodPressure**: pressão arterial diastólica
- **SkinThickness**: dobra da pele do tríceps em milímetros
- **Insulin**: taxa de insulinas
- **BMI**: índice de massa corpórea (IMC)
- **DiabetesPedigreeFunction**: probabilidade de diabetes com base no histórico familiar
- **Age**: idade
- **Outcome**: classe de saída, sendo 0 (sem diabetes) ou 1 (com diabetes)

Todo o código está comentado, para facilitar o entendimento. Vamos iniciar executando um código para que os *warnings* não sejam exibidos e, em seguida, vamos importar os pacotes que utilizaremos neste *notebook*:




```
1 # configuração para não exibir os warnings
2 import warnings
3 warnings.filterwarnings("ignore")
4
5 # Imports necessários
6 import pandas as pd
7 import numpy as np
8 import matplotlib.pyplot as plt
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.preprocessing import MinMaxScaler
11 from sklearn.model_selection import train_test_split
12 from sklearn.model_selection import KFold
13 from sklearn.model_selection import StratifiedKFold
14 from sklearn.model_selection import cross_val_score
15 from sklearn.model_selection import GridSearchCV
16 from sklearn.metrics import accuracy_score
17 from sklearn.pipeline import Pipeline
18 from sklearn.linear_model import LogisticRegression
19 from sklearn.tree import DecisionTreeClassifier
20 from sklearn.neighbors import KNeighborsClassifier
21 from sklearn.naive_bayes import GaussianNB
22 from sklearn.svm import SVC
23 from sklearn.ensemble import BaggingClassifier
24 from sklearn.ensemble import RandomForestClassifier
25 from sklearn.ensemble import ExtraTreesClassifier
26 from sklearn.ensemble import VotingClassifier
27 from sklearn.ensemble import AdaBoostClassifier
28 from sklearn.ensemble import GradientBoostingClassifier
```

Agora, vamos carregar o *dataset* a partir de um arquivo disponível *on-line* e exibir as primeiras linhas do *dataset*:

```
1 # Informa a URL de importação do dataset
2 url = "https://raw.githubusercontent.com/tatianaesc/datascience/main/diabetes.csv"
3
4 # Lê o arquivo
5 dataset = pd.read_csv(url, delimiter=',')
6
7 # Mostra as primeiras linhas do dataset
8 dataset.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	04	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Em seguida, vamos realizar a separação do *dataset* nas bases de treino e teste, com o método *holdout*:

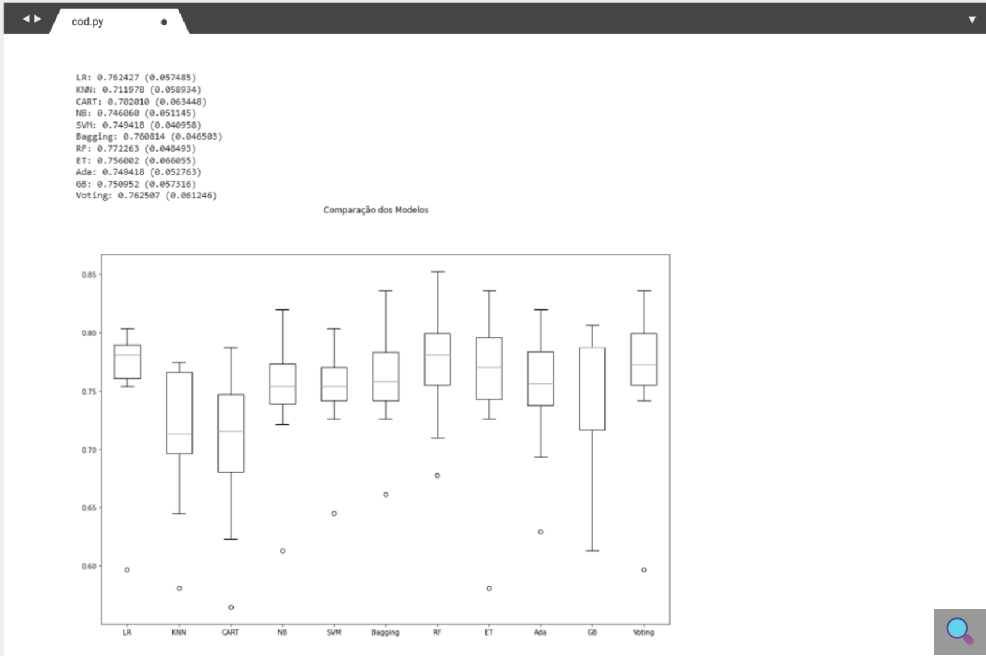
```
1 test_size = 0.20 # tamanho do conjunto de teste
2 seed = 7 # semente aleatória
3
4 # Separação em conjuntos de treino e teste
5 array = dataset.values
6 X = array[:,0:8]
7 y = array[:,8]
8 X_train, X_test, y_train, y_test = train_test_split(X, y,
9     test_size=test_size, shuffle=True, random_state=seed, stratify=y) # holdout com estratificação
10
11 # Parâmetros e partições da validação cruzada
12 scoring = 'accuracy'
13 num_particoes = 10
14 kfold = StratifiedKFold(n_splits=num_particoes, shuffle=True, random_state=seed) # validação cruzada com estratificação
```

Iniciaremos agora a etapa de modelagem e inferência, mas não sabemos de antemão quais algoritmos performarão bem nesse conjunto de dados. Assim, usaremos a validação cruzada para treinar e avaliar os modelos usando a métrica acurácia. Primeiramente, iremos avaliar os algoritmos com a configuração padrão de hiperparâmetros da scikit-Learn:

```
1 np.random.seed(7) # definindo uma semente global
2
3 # Lista que armazenará os modelos
4 models = []
5
6 # Criando os modelos e adicionando-os na lista de modelos
7 models.append(('LR', LogisticRegression(max_iter=200)))
8 models.append(('KNN', KNeighborsClassifier()))
9 models.append(('CART', DecisionTreeClassifier()))
10 models.append(('NB', GaussianNB()))
11 models.append(('SVM', SVC()))
12
13 # Definindo os parâmetros do classificador base para o BaggingClassifier
14 base = DecisionTreeClassifier()
15 num_trees = 100
16 max_features = 3
17
18 # Criando os modelos para o VotingClassifier
19 bases = []
20 model1 = LogisticRegression(max_iter=200)
21 bases.append(('logistic', model1))
22 model2 = DecisionTreeClassifier()
23 bases.append(('cart', model2))
24 model3 = SVC()
25 bases.append(('svm', model3))
26
```

```
cod.py

27 # Criando os ensembles e adicionando-os na lista de modelos
28 models.append(('Bagging', BaggingClassifier(base_estimator=base, n_estimators=num_trees)))
29 models.append(('RF', RandomForestClassifier(n_estimators=num_trees, max_features=max_features)))
30 models.append(('ET', ExtraTreesClassifier(n_estimators=num_trees, max_features=max_features)))
31 models.append(('Ada', AdaBoostClassifier(n_estimators=num_trees)))
32 models.append(('GB', GradientBoostingClassifier(n_estimators=num_trees)))
33 models.append(('Voting', VotingClassifier(bases)))
34
35 # Listas para armazenar os resultados
36 results = []
37 names = []
38
39 # Avaliação dos modelos
40 for name, model in models:
41     cv_results = cross_val_score(model, X_train, y_train, cv=kfold, scoring=scoring)
42     results.append(cv_results)
43     names.append(name)
44     msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std())
45     print(msg)
46
47 # Boxplot de comparação dos modelos
48 fig = plt.figure(figsize=(15,10))
49 fig.suptitle('Comparação dos Modelos')
50 ax = fig.add_subplot(111)
51 plt.boxplot(results)
52 ax.set_xticklabels(names)
53 plt.show()
```



Em seguida, vamos repetir esse processo, utilizando a biblioteca **Pipeline** para criar e avaliar os modelos através da validação cruzada com os dados padronizados e normalizados (evitando o *data leakage*) e comparar o seu resultado com os modelos treinados com o *dataset* original:

```
cod.py

1 np.random.seed(7) # definindo uma semente global para este bloco
2
3 # Listas para armazenar os armazenar os pipelines e os resultados para todas as visões do dataset
4 pipelines = []
5 results = []
6 names = []
7
8
9 # Criando os elementos do pipeline
10
11 # Algoritmos que serão utilizados
12 reg_log = ('LR', LogisticRegression(max_iter=200))
13 knn = ('KNN', KNeighborsClassifier())
14 cart = ('CART', DecisionTreeClassifier())
15 naive_bayes = ('NB', GaussianNB())
16 svm = ('SVM', SVC())
17 bagging = ('Bag', BaggingClassifier(base_estimator=base, n_estimators=num_trees))
18 random_forest = ('RF', RandomForestClassifier(n_estimators=num_trees, max_features=max_features))
19 extra_trees = ('ET', ExtraTreesClassifier(n_estimators=num_trees, max_features=max_features))
20 adaboost = ('Ada', AdaBoostClassifier(n_estimators=num_trees))
21 gradient_boosting = ('GB', GradientBoostingClassifier(n_estimators=num_trees))
22 voting = ('Voting', VotingClassifier(bases))
23
24 # Transformações que serão utilizadas
25 standard_scaler = ('StandardScaler', StandardScaler())
26 min_max_scaler = ('MinMaxScaler', MinMaxScaler())
27
28
```

```
cod.py

29 # Montando os pipelines
30
31 # Dataset original
32 pipelines.append(('LR-orig', Pipeline([reg_log])))
33 pipelines.append(('KNN-orig', Pipeline([knn])))
34 pipelines.append(('CART-orig', Pipeline([cart])))
35 pipelines.append(('NB-orig', Pipeline([naive_bayes])))
36 pipelines.append(('SVM-orig', Pipeline([svm])))
37 pipelines.append(('Bag-orig', Pipeline([bagging])))
38 pipelines.append(('RF-orig', Pipeline([random_forest])))
39 pipelines.append(('ET-orig', Pipeline([extra_trees])))
40 pipelines.append(('Ada-orig', Pipeline([adaboost])))
41 pipelines.append(('GB-orig', Pipeline([gradient_boosting])))
42 pipelines.append(('Vot-orig', Pipeline([voting])))
43
44 # Dataset Padronizado
45 pipelines.append(('LR-padr', Pipeline([standard_scaler, reg_log])))
46 pipelines.append(('KNN-padr', Pipeline([standard_scaler, knn])))
47 pipelines.append(('CART-padr', Pipeline([standard_scaler, cart])))
48 pipelines.append(('NB-padr', Pipeline([standard_scaler, naive_bayes])))
49 pipelines.append(('SVM-padr', Pipeline([standard_scaler, svm])))
50 pipelines.append(('Bag-padr', Pipeline([standard_scaler, bagging])))
51 pipelines.append(('RF-padr', Pipeline([standard_scaler, random_forest])))
52 pipelines.append(('ET-padr', Pipeline([standard_scaler, extra_trees])))
53 pipelines.append(('Ada-padr', Pipeline([standard_scaler, adaboost])))
54 pipelines.append(('GB-padr', Pipeline([standard_scaler, gradient_boosting])))
55 pipelines.append(('Vot-padr', Pipeline([standard_scaler, voting])))
56
```

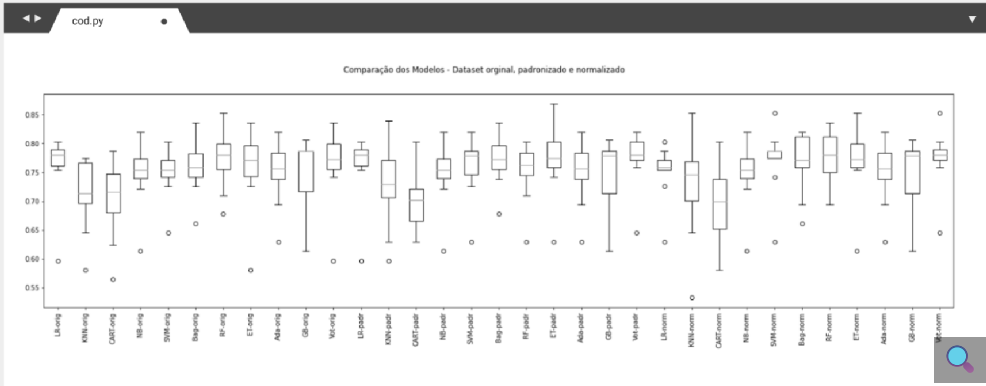


```
cod.py

57 # Dataset Normalizado
58 pipelines.append(('LR-norm', Pipeline([min_max_scaler, reg_log])))
59 pipelines.append(('KNN-norm', Pipeline([min_max_scaler, knn])))
60 pipelines.append(('CART-norm', Pipeline([min_max_scaler, cart])))
61 pipelines.append(('NB-norm', Pipeline([min_max_scaler, naive_bayes])))
62 pipelines.append(('SVM-norm', Pipeline([min_max_scaler, svm])))
63 pipelines.append(('Bag-norm', Pipeline([min_max_scaler, bagging])))
64 pipelines.append(('RF-norm', Pipeline([min_max_scaler, random_forest])))
65 pipelines.append(('ET-norm', Pipeline([min_max_scaler, extra_trees])))
66 pipelines.append(('Ada-norm', Pipeline([min_max_scaler, adaboost])))
67 pipelines.append(('GB-norm', Pipeline([min_max_scaler, gradient_boosting])))
68 pipelines.append(('Vot-norm', Pipeline([min_max_scaler, voting])))
69
70 # Executando os pipelines
71 for name, model in pipelines:
72     cv_results = cross_val_score(model, X_train, y_train, cv=kfold, scoring=scoring)
73     results.append(cv_results)
74     names.append(name)
75     msg = "%s: %.3f (%.3f)" % (name, cv_results.mean(), cv_results.std()) # formatando para 3 casas decimais
76     print(msg)
77
78 # Boxplot de comparação dos modelos
79 fig = plt.figure(figsize=(25,6))
80 fig.suptitle("Comparação dos Modelos - Dataset original, padronizado e normalizado")
81 ax = fig.add_subplot(111)
82 plt.boxplot(results)
83 ax.set_xticklabels(names, rotation=90)
84 plt.show()
```

```
cod.py

LR-orig: 0.762 (0.057)
KNN-orig: 0.712 (0.059)
CART-orig: 0.702 (0.063)
NB-orig: 0.746 (0.051)
SVM-orig: 0.748 (0.041)
Bag-orig: 0.761 (0.047)
RF-orig: 0.772 (0.048)
ET-orig: 0.756 (0.066)
Ada-orig: 0.749 (0.053)
GB-orig: 0.751 (0.057)
Vot-orig: 0.763 (0.061)
LR-padr: 0.762 (0.057)
KNN-padr: 0.727 (0.069)
CART-padr: 0.699 (0.048)
NB-padr: 0.746 (0.051)
SVM-padr: 0.763 (0.053)
Bag-padr: 0.772 (0.045)
RF-padr: 0.751 (0.048)
ET-padr: 0.776 (0.061)
Ada-padr: 0.749 (0.053)
GB-padr: 0.748 (0.058)
Vot-padr: 0.774 (0.047)
LR-norm: 0.751 (0.045)
KNN-norm: 0.725 (0.083)
CART-norm: 0.697 (0.069)
NB-norm: 0.746 (0.051)
SVM-norm: 0.772 (0.054)
Bag-norm: 0.766 (0.050)
RF-norm: 0.776 (0.043)
ET-norm: 0.767 (0.059)
Ada-norm: 0.749 (0.053)
GB-norm: 0.749 (0.058)
Vot-norm: 0.774 (0.049)
```



Agora iremos utilizar a biblioteca **Grid Search** para executar a otimização dos hiperparâmetros do algoritmo KNN, a título de exemplificação, a fim de buscar resultados ainda melhores. Para tal, tentaremos todos os valores ímpares de k entre 1 e 21 e diferentes métricas de distância. Você pode experimentar também fazer a otimização de outros algoritmos para verificar se é possível encontrar uma configuração de modelo que supere os melhores resultados até o momento.

```
cod.py

1 # Tuning do KNN
2
3 np.random.seed(7) # definindo uma semente global para este bloco
4
5 pipelines = []
6
7 # Definindo os componentes do pipeline
8 knn = ('KNN', KNeighborsClassifier())
9 standard_scaler = ('StandardScaler', StandardScaler())
10 min_max_scaler = ('MinMaxScaler', MinMaxScaler())
11
12 pipelines.append(('knn-orig', Pipeline(steps=[knn])))
13 pipelines.append(('knn-padr', Pipeline(steps=[standard_scaler, knn])))
14 pipelines.append(('knn-norm', Pipeline(steps=[min_max_scaler, knn])))
15
16 param_grid = {
17     'KNN__n_neighbors': [1,3,5,7,9,11,13,15,17,19,21],
18     'KNN__metric': ["euclidean", "manhattan", "minkowski"],
19 }
20
21 # Prepara e executa o GridSearchCV
22 for name, model in pipelines:
23     grid = GridSearchCV(estimator=model, param_grid=param_grid, scoring=scoring, cv=kfold)
24     grid.fit(X_train, y_train)
25     # imprime a melhor configuração
26     print("Sem tratamento de missings: %s - Melhor: %f usando %s" % (name, grid.best_score_, grid.best_params_))
27
28 Sem tratamento de missings: knn-orig - Melhor: 0.787615 usando ('KNN__metric': 'manhattan', 'KNN__n_neighbors': 17)
29 Sem tratamento de missings: knn-padr - Melhor: 0.757509 usando ('KNN__metric': 'manhattan', 'KNN__n_neighbors': 19)
30 Sem tratamento de missings: knn-norm - Melhor: 0.799228 usando ('KNN__metric': 'euclidean', 'KNN__n_neighbors': 13)
```

Podemos agora finalizar o modelo escolhido. Apenas para exemplificar, suponha que o modelo que alcançou os melhores

resultados e foi o escolhido para implementação foi a regressão logística com os dados padronizados. A seguir, finalizaremos esse modelo, treinando-o em todo o conjunto de dados de treinamento (sem validação cruzada), e faremos predições para o conjunto de dados de teste que foi separado logo no início da prática, a fim de confirmarmos os resultados. Primeiro, iremos realizar a padronização dos dados de entrada. Depois, treinaremos o modelo e exibiremos a acurácia de teste.

```
1 # Avaliação do modelo com o conjunto de testes
2
3 # Preparação do modelo
4 scaler = StandardScaler().fit(X_train) # ajuste do scaler com o conjunto de treino
5 rescaledX = scaler.transform(X_train) # aplicação da padronização no conjunto de treino
6 model = LogisticRegression(max_iter=200)
7 model.fit(rescaledX, y_train)
8
9 # Estimativa da acurácia no conjunto de teste
10 rescaledTestX = scaler.transform(X_test) # aplicação da padronização no conjunto de teste
11 predictions = model.predict(rescaledTestX)
12 print(accuracy_score(y_test, predictions))

0.7727272727272727
```

Por meio do conjunto de teste, verificamos que alcançamos acurácia de 77,27% em dados não vistos. Esse resultado foi ainda melhor do que a nossa avaliação anterior da regressão logística, que alcançou acurácia média de 76,2%. Valores semelhantes à acurácia de teste são esperados quando esse modelo estiver executando em produção e fazendo predições para novos dados.

Vamos agora preparar o modelo para utilização em produção. Para isso, vamos treiná-lo com todo o *dataset*, e não apenas o conjunto de treino.

```
1 # Preparação do modelo com TODO o dataset
2 scaler = StandardScaler().fit(X) # ajuste do scaler com TODO o dataset
3 rescaledX = scaler.transform(X) # aplicação da padronização com TODO o dataset
4 model.fit(rescaledX, y)

LogisticRegression(max_iter=200)
```

Finalmente, iremos simular a aplicação do modelo em dados não vistos, imaginando que chegaram três novas instâncias, mas que não sabemos a classe de saída. Poderemos, então, aplicar nosso modelo recém-treinado para fazer a predição das classes. Para tal, será necessário antes padronizar os dados utilizando a mesma escala dos dados usados no treinamento do modelo:

```
1 # Novos dados - não sabemos a classe!
2 data = {'preg': [1, 9, 5],
3         'plas': [90, 100, 110],
4         'pres': [50, 60, 50],
5         'skin': [30, 30, 30],
6         'test': [100, 100, 100],
7         'mass': [20.0, 30.0, 40.0],
8         'pedi': [1.0, 2.0, 1.0],
9         'age': [15, 40, 40],
10        }
11
12 atributos = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age']
13 entrada = pd.DataFrame(data, columns=atributos)
14
15 array_entrada = entrada.values
16 X_entrada = array_entrada[:,0:8].astype(float)
17
18 # Padronização nos dados de entrada usando o scaler utilizado em X
19 rescaledEntradaX = scaler.transform(X_entrada)
20 print(rescaledEntradaX)

[[-0.84488505 -0.96601063 -0.90770975  0.59362962  0.17539902 -1.52208097
  1.59499624 -1.55207990]
 [ 1.53004665 -0.65393918 -0.47073225  0.59362962  0.17539902 -0.25269651
  4.62511492  0.57511787]
 [ 0.3429808  -0.34066773 -0.90770975  0.59362962  0.17539902  1.01629594
  1.59499624  0.57511787]]
```

Podemos agora realizar a predição das classes para os novos dados:

```
1 # Predição de classes dos dados de entrada
2 saidas = model.predict(rescaledEntradaX)
3 print(saidas)

[0. 1. 1.]
```



React de mercado

O vídeo a seguir traz o depoimento do profissional Adriano Koshiyama, que falará sobre alguns desafios práticos de projetos de machine learning no mundo real.



Referências

Clique aqui para acessar as referências e créditos desta aula.



BISHOP, C. M.; NASRABADI, N. M. **Pattern recognition and machine learning**. New York: Springer, 2006.

BURKOV, A. **The hundred-page machine learning book**. Québec City: Andriy Burkov, 2019.

ESCOVEDO, T.; KOSHIYAMA, A. **Introdução a data science**: algoritmos de *machine learning* e métodos de análise. São Paulo: Casa do Código, 2020.

GÉRON, A. **Mãos à obra**: aprendizado de máquina com scikit-learn, Keras & TensorFlow – conceitos, ferramentas e técnicas para a construção de sistemas inteligentes. Sebastopol: O’Reilly Media, 2021.

KALINOWSKI, M.; ESCOVEDO, T.; VILLAMIZAR, H.; LOPES, H. **Engenharia de software para ciência de dados**: um guia de boas práticas com ênfase na construção de sistemas de *machine learning* em python. São Paulo: Casa do Código, 2023.

MURPHY, K. P. **Machine learning**: a probabilistic perspective. Cambridge: MIT Press, 2012.

VANDERPLAS, J. **Python data science handbook**: essential tools for working with data. Sebastopol: O'Reilly Media, 2016.

Bancos de imagens Pexels, Pixabay, Envato, Freepik e Shutterstock.